

Αλέξανδρος Γκιάφης sdi2200284

Άσκηση 2.1

Όνομα υπολογιστικού συστήματος: linux07 της σχολής

Μοντέλο επεξεργαστή: Intel(R) Core(TM) i5-6500 CPU @ 3.20GHz

Αριθμός πυρήνων: 4

Threads ανα πυρήνα: 1

Έκδοση λειτουργικού συστήματος: Ubuntu 20.04.6 LTS

Έκδοση μεταγλωττιστή: gcc (Ubuntu 9.4.0-1ubuntu1~20.04.2) 9.4.0

Στην παρούσα εργασία, ζητήθηκε να αναπτύξω ένα πρόγραμμα σε C που προσομοιώνει το παιχνίδι της ζωής (Game of Life). Το παιχνίδι βασίζεται σε ένα πλέγμα που ξεκινά από μια αρχική κατάσταση, όπου ορισμένα κελιά είναι ζωντανά και άλλα όχι. Στη συνέχεια, προκύπτουν επόμενες γενιές με βάση ένα σύνολο απλών κανόνων που καθορίζουν:

- πότε ένα ζωντανό κελί πεθαίνει,
- πότε ένα ζωντανό κελί παραμένει ζωντανό,
- και πότε ένα μη ζωντανό κελί «γεννιέται».

Είσοδος του Προγράμματος

Το πρόγραμμα δέχεται ως είσοδο:

1. Τον αριθμό των γενιών που θα προσομοιωθούν,
2. Το μέγεθος του πλέγματος,
3. Τον τύπο εκτέλεσης (σειριακή ή παράλληλη),
4. Τον αριθμό των νημάτων που θα χρησιμοποιηθούν.

Σκοπός της Εργασίας

Ο κύριος στόχος ήταν η παραλληλοποίηση του προγράμματος με τη χρήση του OpenMP, ώστε να επιτευχθεί αποτελεσματική κατανομή της επεξεργασίας στα διαθέσιμα νήματα και να μεγιστοποιηθεί η απόδοση.

Υλοποίηση

Η παραλληλοποίηση υλοποιήθηκε με τη δημιουργία μιας ομάδας από threads, όπου κάθε thread αναλαμβάνει να επεξεργαστεί ένα τμήμα του πλέγματος. Σε κάθε φάση της προσομοίωσης:

1. Κάθε thread σαρώνει το τμήμα του πλέγματος που του έχει ανατεθεί και υπολογίζει την κατάσταση του νέου πλέγματος.
2. Για να αποφύγω race conditions και περιττό συγχρονισμό, χρησιμοποίησα **ιδιωτικές (private)** μεταβλητές για κάθε thread. Αυτές περιλαμβάνουν προσωρινές δομές (**tempBoard**, **currentBoard**, και **newBoard**), οι οποίες ενημερώνονται τοπικά από κάθε thread.

3. Ο συγχρονισμός μεταξύ των threads εξασφαλίζεται από το OpenMP μέσω του **implicit barrier** στο τέλος του **for** βρόχου, διασφαλίζοντας ότι όλα τα threads ολοκληρώνουν την επεξεργασία πριν από τη μετάβαση στην επόμενη γενιά.

Λεπτομέρειες Υλοποίησης

Στον κώδικα:

- Τα threads χρησιμοποιούν την οδηγία **#pragma omp parallel** για τη δημιουργία της ομάδας.
- Η επεξεργασία κάθε γενιάς γίνεται μέσω της οδηγίας **#pragma omp for collapse(2)**, που διασπά το πλέγμα σε μικρότερα κομμάτια και τα αναθέτει στα threads.
- Η εναλλαγή μεταξύ του τρέχοντος πλέγματος (**currentBoard**) και του νέου πλέγματος (**newBoard**) γίνεται με τη χρήση μιας προσωρινής μεταβλητής (**tempBoard**) και είναι πλήρως ιδιωτική για κάθε thread, αποφεύγοντας έτσι τυχόν συγκρούσεις.

Στο τέλος της προσομοίωσης, ένας μόνο thread, μέσω της οδηγίας **#pragma omp single**, ενημερώνει τις κοινές δομές δεδομένων (**currentBoard** και **newBoard**), διασφαλίζοντας έτσι την ακεραιότητα των δεδομένων.

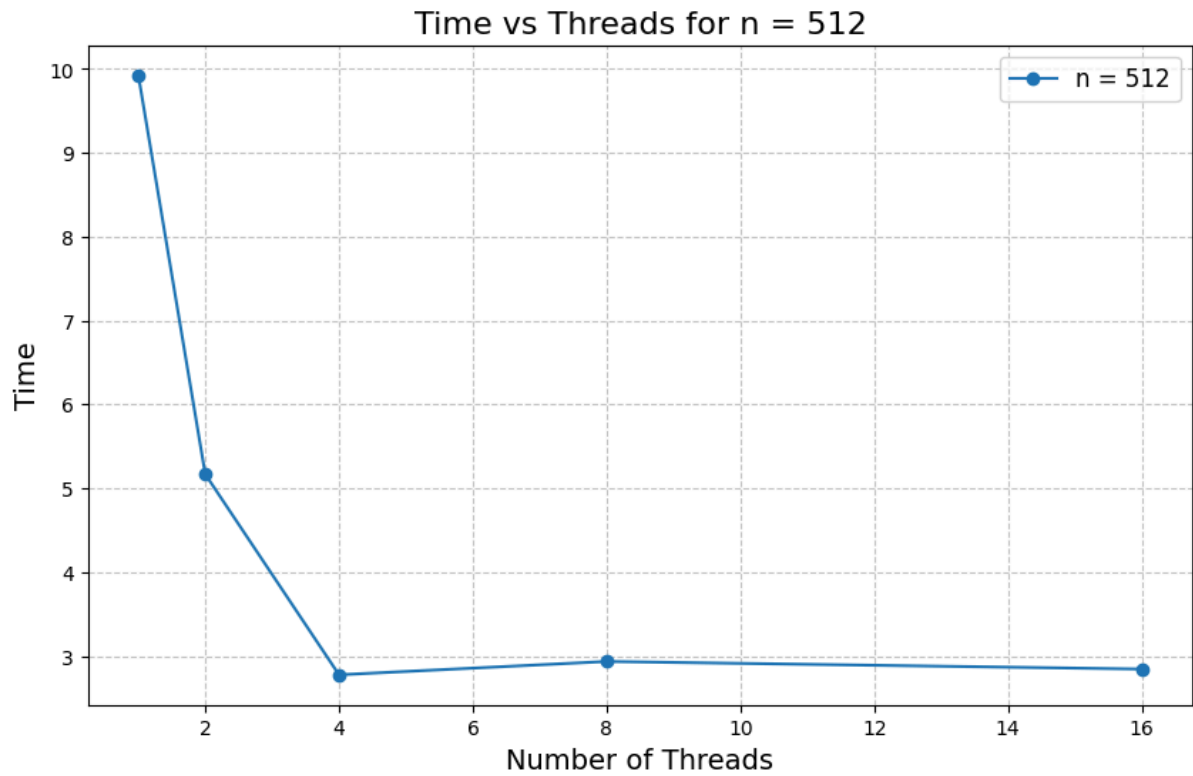
Δοκιμές

Για τη δοκιμή του προγράμματος, εκτέλεσα προσομοιώσεις 1000 γενιών σε διαφορετικές συνθήκες. Συγκεκριμένα, πειραματίστηκα με:

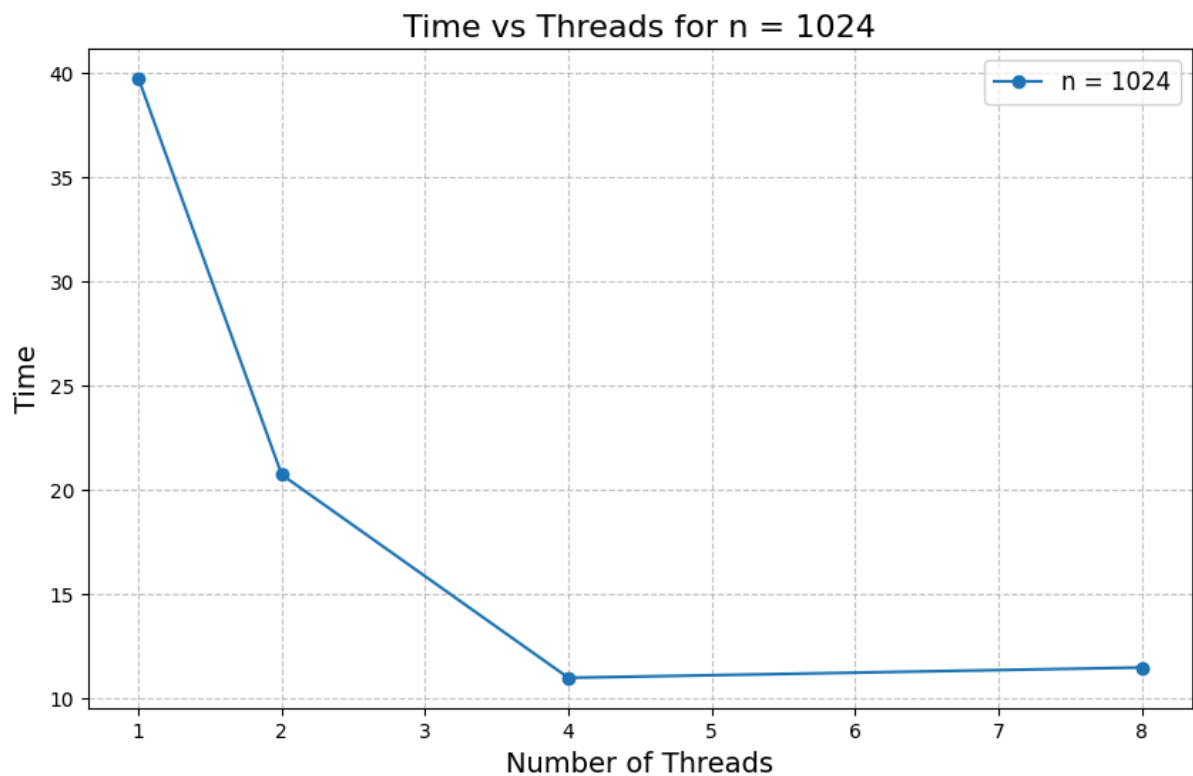
- Διάφορα μεγέθη πλέγματος,
- Αριθμούς νημάτων,

Στα παραπάνω διαγράμματα απεικονίζεται ο χρόνος που απαιτείται για τον υπολογισμό της τελικής γενιάς, εκτελώντας την προσομοίωση για 1000 γενιές σε ένα πλέγμα συγκεκριμένης διάστασης n . Στον οριζόντιο άξονα (x) παρουσιάζεται ο αριθμός των νημάτων, ενώ στον κατακόρυφο άξονα (y) καταγράφεται ο χρόνος ολοκλήρωσης της προσομοίωσης.

Πλέγμα διάστασης 512 x 512:



Πλέγμα διάστασης 1024×1024 :



Να σημειωθεί ότι για $\text{threads}=1$ εκτελείται ο σειριακός αλγόριθμος, ενώ για $\text{threads} > 1$ εκτελείται ο παράλληλος. Όπως φαίνεται από τα αποτελέσματα, καθώς αυξάνονται τα νήματα, ο χρόνος εκτέλεσης μειώνεται σχεδόν στο μισό. Συγκεκριμένα, για 2 νήματα ο

χρόνος είναι σχεδόν μισός σε σχέση με τον σειριακό αλγόριθμο, για 4 νήματα ο χρόνος μειώνεται κατά το ήμισυ σε σχέση με τα 2 νήματα, ενώ για περισσότερα από 4 νήματα ο χρόνος παραμένει σταθερός όπως ακριβώς ήταν για 4. Αυτό συμβαίνει διότι το σύστημα διαθέτει μόνο 4 πυρήνες, οπότε το πολύ 4 νήματα μπορούν να τρέχουν ταυτόχρονα, με καθένα να αναλαμβάνει ένα thread.

Αν και κάποιος ίσως περίμενε ότι ο χρόνος θα αυξανόταν για threads > 4 λόγω του γεγονότος ότι κάποια threads πρέπει να περιμένουν, κάτι τέτοιο δεν συμβαίνει. Ο λόγος είναι ότι, μόλις ένα thread ολοκληρώσει το κομμάτι του, μπορεί να αναλάβει άλλο thread για να ξεκινήσει το δικό του. Έτσι, καθ' όλη τη διάρκεια της εκτέλεσης θα παραμένουν ενεργά 4 threads ταυτόχρονα. Είτε έχουμε 4 νήματα που δουλεύουν σε 4 μεγάλα τμήματα του πίνακα, είτε 8 νήματα που επεξεργάζονται 8 μικρότερα τμήματα, ο συνολικός φόρτος εργασίας παραμένει ίδιος. Η παραλληλία διατηρείται και η επίδραση του overhead από την εναλλαγή των threads και τη δημιουργία νέων νημάτων είναι μικρή και σχεδόν αμελητέα.